

E-BiGraphSAGE: Bipartite Graph Neural Networks for Network Intrusion Detection

1st Qinxin Zhao

National Key Lab for Novel Software Technology
Nanjing University
Nanjing, China
652024320007@smail.nju.edu.cn

2nd Sheng Zhong*

National Key Lab for Novel Software Technology
Nanjing University
Nanjing, China
zhongsheng@nju.edu.cn

Abstract—The frequent impacts and serious damages of cyberattacks raise an urgent need for robust network intrusion detection amidst escalating cyber threats. Traditional signature-based methods often fail to detect novel attacks, necessitating more adaptive anomaly-based approaches. To address the challenge of effective network traffic modeling, we propose a novel method named E-BiGraphSAGE, which captures intricate relationships between source and destination addresses of network traffic activities in a bipartite graph. The proposed method comprises three main components: flow-centric bipartite graph construction, backtracking bipartite graph learning, and vertex splitting strategy. Firstly, a flow-centric bipartite graph is constructed to better represent the flow-based dataset, effectively capturing the crucial information of network flows. Secondly, a backtracking bipartite graph learning model is introduced to learn the representations of heterogeneous vertices within this flow-centric bipartite graph. Furthermore, a vertex splitting strategy is implemented to reduce the computational complexity of the method. Empirical evaluations on four benchmark NIDS datasets demonstrate that E-BiGraphSAGE outperforms state-of-the-art methods in terms of detection accuracy and robustness, offering a more effective solution for network intrusion detection.

Index Terms—Graph Neural Networks, Network Intrusion Detection, Bipartite Graph.

I. INTRODUCTION

Cyberattacks are malicious activities to illegally access, alter or destroy data by intruding into network systems, such as DoS (Denial of Service) attacks [1] and reconnaissance attacks [2]. With the rapid development of Internet around the world, cyberattacks are making more frequent impacts and posing more serious damages. It is reported that 91% of organizations are experiencing cybersecurity incident annually [3], and the global cost is expected to reach 23.84 trillion dollars by 2027 [4]. Therefore, it is an urgent task to detect different kinds of intrusions and protect the network security from cyberattacks.

As a research hotspot in the field of cybersecurity, network intrusion detection systems (NIDS) have received extensive attention from researchers. Traditional NIDS methods are mainly based on explicit signature and heuristic rules predefined in the database [5]–[7]. These signature-based methods work well at identifying well-known attacks with exclusive patterns, but may fail to detect some suspicious behaviours

*Sheng Zhong is the corresponding author.

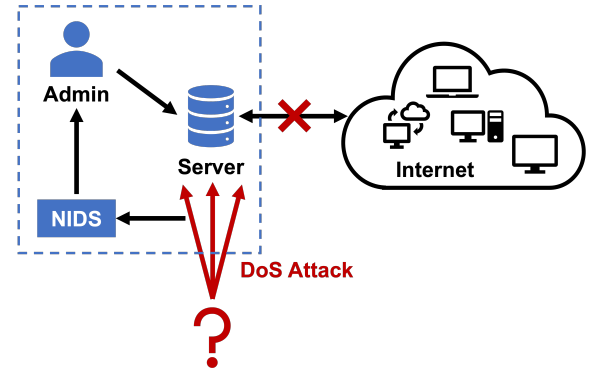


Fig. 1: Illustration of the important role of network intrusion detection system (NIDS) in the cybersecurity domain. NIDS is designed to collect and monitor traffic features within the network system. When an attack (like DoS attack) occurs, NIDS can automatically detect the potential anomaly behaviours and alert the system administrator to take corresponding actions.

from novel attacks like zero-day attacks [8]. To address this problem, anomaly-based NIDS methods are proposed to learn the implicit patterns of network traffics from observed data with machine learning techniques, such as support vector machine [9], naive bayes [10], decision tree [11] and so on. With the development of deep learning in computer vision [12], [13], deep neural networks are also introduced to automatically learn the latent features of network traffic from large NIDS datasets [14], [15]. Especially, graph neural networks (GNNs) have attracted increasing attention in NIDS due to their intrinsic efficiency in modeling network traffic topology. Many GNN-based methods have achieved performance improvement in the network intrusion detection task [16]–[18].

Although the GNN-based methods have shown promising prospects, there are still some crucial challenges that prevent them from effective and robust detection of network intrusions. **Firstly**, existing methods often model network traffic activities as an endpoint-centric graph, where the nodes represent the endpoints and the edges represent the connections between endpoints [17], [18]. The endpoint/node-centric graph is straightforward in modeling the network topology but may overlook the significance of network flow/edge for network

intrusion detection [19], [20]. **Secondly**, the heterogeneous relationships between nodes and flows in the network traffic are not well modeled in the existing methods, which requires a suitable graph neural network to learn the heterogeneous representations of the network traffic graph structure. **Additionally**, computational efficiency is another challenge in the network intrusion detection task. Due to the imbalanced traffic distribution in the network topology [21], [22], existing methods may suffer from computational inefficiency and excessive memory when processing the vertexes with extremely high degrees. These challenges may lead to suboptimal performance in the network intrusion detection task.

To address the above challenges, we propose a novel method for network intrusion detection using bipartite graph neural networks, termed E-BiGraphSAGE. This method introduces a unique approach to model network traffic activities as a bipartite graph, effectively capturing the intricate and directed relationships between the source and destination addresses of network flows. By transforming traditional graph learning into a backtracking bipartite graph learning framework, E-BiGraphSAGE enables the effective learning of representations for heterogeneous vertices, including flow vertices and endpoint vertices, within the bipartite graph. This allows the model to better understand the complex interactions and dependencies in network traffic, which are critical for accurately identifying both normal and malicious activities. To further enhance the computational efficiency of the algorithm, a vertex splitting strategy is introduced. This strategy addresses the challenges posed by high-degree nodes, which are common in real-world network traffic, by reducing computational overhead and preventing information bottlenecks during the learning process. As a result, the proposed method not only improves scalability but also ensures that the model remains efficient when applied to large-scale datasets. Through extensive empirical evaluations on four widely recognized network intrusion detection datasets demonstrates superior performance compared to state-of-the-art methods. The results validate the effectiveness and practicality of E-BiGraphSAGE as a comprehensive solution for modern network intrusion detection tasks.

The main contributions of this paper are as follows:

- We propose a novel method, E-BiGraphSAGE, for network intrusion detection using bipartite graph neural networks. The proposed method effectively models network traffic activities as a bipartite graph, capturing the intricate relationships between source and destination addresses of network flows.
- We introduce a backtracking bipartite graph learning model with vertex splitting strategy to learn the structural information of heterogeneous vertices in the network traffic data, which can effectively and efficiently capture the directed relationships of network flows.
- We conduct extensive experiments on real-world network intrusion detection datasets, and the results show that the proposed method outperforms the state-of-the-art methods in terms of detection accuracy and robustness.

II. RELATED WORK

A. Graph Neural Networks

Graph Neural Networks (GNN) are a class of neural networks tailored for processing data with graph structures [23]. They have become increasingly prominent for their ability to capture the complex patterns inherent in the interconnected data points of a graph. Owing to its wide success [24], deep neural networks have been extensively studied in various tasks including node classification, link prediction, graph classification, and graph generation.

In the past few years, the field of GNNs has witnessed significant progress. Early works in GNNs focused on developing models for normal graph. The Graph Convolutional Network (GCN) [25] introduced a scalable approach for semi-supervised node classification by effectively aggregating information from neighboring nodes in the graph. GraphSAGE [26] introduced a framework for inductive representation learning on large-scale graphs by sampling and aggregating features from a node's local neighborhood. Graph Attention Network (GAT) [27] improved upon GCN by introducing attention mechanisms to weight the importance of neighbor nodes during aggregation, leading to enhanced expressive power. These traditional GNN backbone models have been widely adopted in network anomaly detection [16]–[18], recommendation systems [28]–[31], and so on.

In addition to the normal graph, GNN have also been extended to handle more complex graph structures, such as bipartite graphs. Bipartite graphs is a type of graph where nodes can be divided into two disjoint sets such that no two nodes within the same set are adjacent. The unique structural properties of bipartite graphs have motivated the development of specialized GNN models in many applications like recommendation systems [32], [33] and medical informatics [34], [35]. However, it is still an open question to explore how to effectively leverage the structural information of bipartite graphs to enhance the performance of network intrusion detection systems.

B. Graph-based Network Intrusion Detection

Network intrusion detection is a critical problem in cybersecurity to protect network systems from cyberattacks. Existing network intrusion detection systems (NIDS) can be mainly categorized into two types: signature-based approaches [5]–[7] and anomaly-based approaches [9], [11], [17]. The anomaly-based approaches generally regard this problem as a classification task in the machine learning paradigm. Early anomaly-based works are mainly focused on designing handcrafted traffic features with support vector machine [9], naive bayes [10], decision tree [11] and so on. With the development of deep learning, deep neural networks are introduced to automatically learn the latent features of network traffic from large NIDS dataset, including convolutional neural networks (CNN) [14], recurrent neural networks (RNN) [15], and so on.

Recently, graph learning have attracted increasing attention in NIDS due to its intrinsic efficiency in modeling network

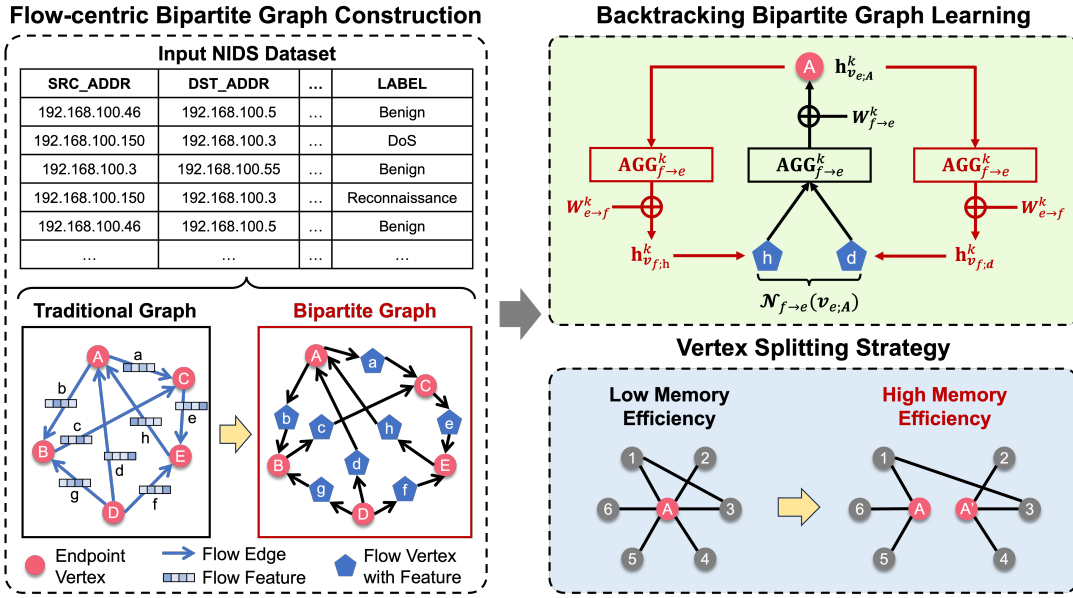


Fig. 2: The main framework of the proposed E-BiGraphSAGE for flow-based network intrusion detection. The proposed method consists of three components: the flow-centric bipartite graph construction, the backtracking bipartite graph learning, and the vertex splitting strategy.

traffic topology. Many graph-based methods are proposed for network intrusion detection task, such as E-ResGAT [16], E-GraphSAGE [17], and Anomal-E [18]. E-ResGAT [16] integrates the powerful Graph Attention Networks (GAT) with residual learning to enhance intrusion detection, which helps to effectively deal with class imbalances problem in cybersecurity datasets. E-GraphSAGE [17] incorporates both edge features and topological information of network flows into the learning process of traditional GraphSAGE, which allows more effective extraction of the nuanced patterns indicative of malicious activities within IoT networks. Anomal-E [18] employs a modified version of the E-GraphSAGE model, which introduces a deep graph infomax method for self-supervised learning to maximize the mutual information between local edge information and the global graph structure, enabling the model to learn more robust embeddings.

Existing graph-based network intrusion detection methods generally model the flow-based NIDS dataset as an ordinary graph structure, regarding the endpoints as the nodes and the traffic flows as the edges. This modeling paradigm only captures the connections between the endpoints, but fails to explicitly describe the connections among the network traffic edges, which may lead to suboptimal performance in the NIDS task. Thus, it is necessary to explore more effective graph learning models to capture the heterogeneous relationships between the network traffic flows and the endpoints in the NIDS dataset.

III. PROBLEM FORMULATION

Given a flow-based NIDS dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ with an endpoint index set $\mathcal{P} = \{1, \dots, k\}$, where $x_i = (s_i, d_i, z_i)$ is the flow record and y_i is the class label of the i -th network

flow. The flow record x_i is further composed of the source endpoint index $s_i \in \mathcal{P}$, the destination endpoint index $d_i \in \mathcal{P}$, and the m -dimensional feature vector $z_i \in \mathbb{R}^m$ of the i -th network flow. The goal of the network intrusion detection task is to learn a mapping function $f : (s, d, z) \mapsto y$ that can accurately predict the class label of the network flow.

IV. PROPOSED METHOD

A. Method Overview

As shown in Fig. 2, the proposed E-BiGraphSAGE method consists of three main components: the flow-centric bipartite graph construction, the backtracking bipartite graph learning, and the splitting strategy. **Firstly**, a flow-centric bipartite graph is constructed to represent the flow-based dataset in a more suitable manner, which can effectively capture the key information of network flows. **Secondly**, a backtracking bipartite graph learning model is designed to learn the representations of heterogeneous vertices in the flow-centric bipartite graph, which can effectively capture the directed relationships of network flows. **Thirdly**, a vertex splitting strategy is proposed to split the vertex with a large number of connections into multiple vertices, which can effectively reduce the computational complexity of the proposed method. The details of the proposed method are as follows.

B. Flow-centric Bipartite Graph Construction

In the existing GNN-based NIDS methods [16]–[18], the flow-based dataset is usually modeled as an endpoint-centric graph to feed into the GNNs. For example, E-GraphSAGE [17] models the flow-based dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where the vertex set $\mathcal{V} = \{v_1, \dots, v_k\}$ denotes

the k endpoints from $\mathcal{P}$ and the edge set $\mathcal{E} = \{e_1, \dots, e_n\}$ denotes the n records from $\mathcal{D}$. Each edge $e_i \in \mathcal{E}$ corresponds to a connection between the vertex v_{s_i} and v_{d_i} with z_i as the edge feature. The graph $\mathcal{G}$ is then fed into a modified GraphSAGE model to learn the edge representations of different traffic flows.

However, the above endpoint-centric graph makes it difficult for GNNs to capture the key information in the flow-based dataset. In the constructed graph $\mathcal{G}$, the key information of traffic flows z_i are modeled as the edge features, which are not well incorporated in the popular GNN models like GCN and GAT [20]. This problem leads to the inapplicability of a large scale of GNN models in the flow-based NIDS dataset, and may lead to suboptimal performance in the NIDS task.

To address this issue, we propose a flow-centric bipartite graph to model the flow-based dataset in a more suitable manner. As shown in Fig. 2, the flow-centric bipartite graph is constructed as:

$$\mathcal{G}_b = (\mathcal{V}_e, \mathcal{V}_f, \mathcal{E}_{e \rightarrow f}, \mathcal{E}_{f \rightarrow e}) \quad (1)$$

where the vertex set consists of two disjoint sets of vertices: the endpoint set $\mathcal{V}_e = \{v_{e;1}, \dots, v_{e;k}\}$ from k endpoints in $\mathcal{P}$, and the flow set $\mathcal{V}_f = \{v_{f;1}, \dots, v_{f;n}\}$ from n records in $\mathcal{D}$ with the corresponding input features $z_1, \dots, z_n$. The edge set also consists of two disjoint sets of directed edges $\mathcal{E}_{e \rightarrow f}, \mathcal{E}_{f \rightarrow e}$ from the endpoints to flow and vice versa, which are defined as:

$$\mathcal{E}_{e \rightarrow f} = \{(v_{e;s_i}, v_{f;d_i}) | (s_i, d_i, z_i) \in \mathcal{D}\} \quad (2)$$

$$\mathcal{E}_{f \rightarrow e} = \{(v_{f;j}, v_{e;d_i}) | (s_i, d_i, z_i) \in \mathcal{D}\} \quad (3)$$

C. Backtracking Bipartite Graph Learning

After constructing the flow-centric bipartite graph to represent the flow-based dataset, the next challenge for the method is to build a bipartite graph neural network model that can effectively extract the representations of heterogeneous vertex, especially the flow vertex, from the input directed bipartite graph. However, existing graph learning methods, whether general methods or NIDS approaches, are mainly designed for ordinary graph structures, as introduced in Related work. These methods are not directly applicable to the flow-centric bipartite graph, which is a directed and heterogeneous graph structure.

To address this problem, we propose a novel bipartite graph neural network to learn the representations of vertices in the flow-centric bipartite graph. A two-stage backtracking message passing process is designed on the directed bipartite graph, which can effectively capture the directed relationships of the network traffic flows. The details of the proposed network are as follows.

The proposed network consists of K graph convolutional layers, and each layer performs a 2-hop operation on the flow-centric bipartite graph $\mathcal{G}_b = (\mathcal{V}_e, \mathcal{V}_f, \mathcal{E}_{e \rightarrow f}, \mathcal{E}_{f \rightarrow e})$ defined in Eq. (1). Denote h_v^k as the aggregated information of vertex v at the k -th layer ($k = 0, \dots, K$). At the input layer, the information of the flow vertex $v_{f;i}$ is initialized as $h_{v_{f;i}}^0 = z_i$,

while the information of the endpoint vertex $v_{e;i}$ is initialized as $h_{v_{e;i}}^0 = \mathbf{0}$.

At the k -th layer, the message-passing process of the proposed method is divided into two stages: the flow-to-endpoint stage and the endpoint-to-flow stage. At the flow-to-endpoint stage, we utilize an aggregator function $\text{AGG}_{f \rightarrow e}^k$, such as mean and sum function, to pass the information from flow to endpoint. For each endpoint vertex $v_{e;i} \in \mathcal{V}_e$, the $(k-1)$ -layer features of all flow vertex in its neighborhood is aggregated as:

$$h_{\mathcal{N}_{f \rightarrow e}(v_{e;i})}^k = \text{AGG}_{f \rightarrow e}^k(\{h_{v_f}^{k-1} | \forall v_f \in \mathcal{N}_{f \rightarrow e}(v_{e;i})\}) \quad (4)$$

$$h_{v_{e;i}}^k = \sigma(W_{f \rightarrow e}^k \cdot \text{CONCAT}(h_{v_{e;i}}^{k-1}, h_{\mathcal{N}_{f \rightarrow e}(v_{e;i})}^k)) \quad (5)$$

where $W_{f \rightarrow e}^k$ is the weight of the k -th $f \rightarrow e$ GCNConv layer, $\mathcal{N}_{f \rightarrow e}(v_{e;i})$ denotes the set of flow vertices that has an incoming connection to the endpoint vertex $v_{e;i}$ as:

$$\mathcal{N}_{f \rightarrow e}(v_{e;i}) = \{v_{f;j} | (v_{f;j}, v_{e;i}) \in \mathcal{E}_{f \rightarrow e}\} \quad (6)$$

At the endpoint-to-flow stage, we utilize another aggregator function $\text{AGG}_{e \rightarrow f}^k$ to pass the information from endpoint to flow. For each flow vertex $v_{f;i} \in \mathcal{V}_f$, the k -layer features of all endpoint vertex in its neighborhood is aggregated as:

$$h_{\mathcal{N}_{e \rightarrow f}(v_{f;i})}^k = \text{AGG}_{e \rightarrow f}^k(\{h_{v_e}^k | v_{e;j} \in \mathcal{N}_{e \rightarrow f}(v_{f;i})\}) \quad (7)$$

$$h_{v_{f;i}}^k = \sigma(W_{e \rightarrow f}^k \cdot \text{CONCAT}(h_{v_{f;i}}^{k-1}, h_{\mathcal{N}_{e \rightarrow f}(v_{f;i})}^k)) \quad (8)$$

where $W_{e \rightarrow f}^k$ is the weight of the k -th $e \rightarrow f$ backtracking GCNConv layer, $\mathcal{N}_{e \rightarrow f}(v_{f;i})$ denotes the set of endpoint vertices that has an outgoing connection to the flow vertex $v_{f;i}$ as:

$$\mathcal{N}_{e \rightarrow f}(v_{f;j}) = \{v_{e;i} | (v_{e;i}, v_{f;j}) \in \mathcal{E}_{e \rightarrow f}\} \quad (9)$$

The above two-stage message passing process is repeated for K layers, and the final vertex representations h_v^K are used to predict the class label of the network flow.

D. Vertex Splitting Strategy

In the construction of the flow-centric bipartite graph, the number of connections of each vertex may vary significantly, which may lead to computational inefficiency and excessive memory requirements of the proposed method. To address this issue, we propose a vertex splitting strategy to split the vertex with a large number of connections into multiple vertices, which can effectively reduce the computational complexity of the proposed method.

Let $D(v)$ denote the connectivity degree of vertex v . Specifically, a maximum degree of connectivity D_{max} was established for each node within the graph to maintain computational tractability and prevent excessive computational load. We split nodes with connectivity degrees larger than D_{max} into several vertexes with smaller degrees, which can be described as Algorithm 1.

We imposed an upper limit of about $D_{max} = 5,000$ on the node degrees, which is adjustable depending on the specific

Algorithm 1 Splitting algorithm

Input: $D_{max}, \mathcal{G}_b = (\mathcal{V}_e, \mathcal{V}_f, \mathcal{E}_{e \rightarrow f}, \mathcal{E}_{f \rightarrow e})$

Output: $\mathcal{G}'_b = (\mathcal{V}'_e, \mathcal{V}'_f, \mathcal{E}'_{e \rightarrow f}, \mathcal{E}'_{f \rightarrow e})$

Initialize $\mathcal{V}'_e, \mathcal{V}'_f, \mathcal{E}'_{e \rightarrow f}, \mathcal{E}'_{f \rightarrow e}$ as empty sets.

```
for  $v_e \in \mathcal{V}_e$  do
    new vertex  $v'$ ,  $\mathcal{V}'_e \leftarrow \mathcal{V}'_e \cup \{v'\}$ 
    for  $(v_i, v_j) \in \mathcal{E}_{e \rightarrow f} \cup \mathcal{E}_{f \rightarrow e}$  do
        if  $v_i = v_e$  or  $v_j = v_e$  then
            if  $D(v') \geq D_{max}$  then
                new vertex  $v'$ ,  $\mathcal{V}'_e \leftarrow \mathcal{V}'_e \cup \{v'\}$ 
            end if
            if  $v_i = v_e$  then
                 $\mathcal{E}'_{e \rightarrow f} \leftarrow \mathcal{E}'_{e \rightarrow f} \cup \{(v', v_j)\}$ 
            else if  $v_j = v_e$  then
                 $\mathcal{E}'_{f \rightarrow e} \leftarrow \mathcal{E}'_{f \rightarrow e} \cup \{(v_i, v')\}$ 
            end if
        end if
    end for
end for
```

dataset. In instances where the degree of any given node threatened to exceed this cap, a randomized partitioning process was implemented. This process involved splitting the node in question, such that the combined total of its incoming and outgoing connections would conform to the established threshold. This threshold ensured the preservation of computational efficiency throughout the analysis, without compromising on the integrity of the network structure captured by the bipartite graph representation.

V. EXPERIMENTS

In this section, we evaluate the performance of the proposed E-BiGraphSAGE method on 4 publicly available NIDS datasets. We compare the proposed method with state-of-the-art methods including E-GraphSAGE [17] and E-ResGAT [16]. We also conduct ablation studies to investigate the effectiveness of the proposed method.

A. Experiment Setup

1) *Datasets:* In the experiments, 4 datasets are used to evaluate the performance of the proposed method, including the UNSW-NB15 dataset [37], the CSE-CIC-IDS2018 dataset [38], the BoT-IoT dataset [39], and the ToN-IoT dataset [40]. The details of these datasets are summarized as the followings:

- **UNSW-NB15** dataset was created in 2015 by the Cyber Range Lab at the University of New South Wales (UNSW) and contains 2,540,044 network traffic records with 49 features extracted using Argus and Bro-IDS tools. Each record is labeled as either normal or malicious, with malicious traffic further categorized into 9 attack types, including Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The dataset was generated using the IXIA PerfectStorm tool to simulate realistic network traffic and is widely used

for evaluating intrusion detection systems and network security models.

- **CSE-CIC-IDS2018** was collected in 2018 by the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC) in 2018 and contains 16,232,943 records with 75 features. It was designed to simulate realistic network traffic, including both benign activities and seven types of attack scenarios, such as Brute-force, DoS, DDoS, and Web attacks. The dataset includes raw PCAP files, system logs, and over 80 extracted features, providing a comprehensive resource for evaluating intrusion detection systems. The dataset is labeled with 7 types of network attacks, including 6 types of attacks and 1 type of normal traffic.
- **BoT-IoT** is a comprehensive dataset designed for network forensic analytics in IoT environments developed in the Cyber Range Lab of UNSW Canberra. It includes a mix of normal and botnet traffic, with attack types such as DDoS, DoS, OS and Service Scan, Keylogging, and Data Exfiltration. The dataset contains 3,668,522 records with 42 features. The dataset is freely available for academic research and has been widely cited in studies on intrusion detection, network forensics, and IoT security.
- **ToN-IoT** are a comprehensive collection of Industry 4.0/IoT and Industrial IoT (IIoT) datasets designed for evaluating AI-based cybersecurity applications. It contains 22,339,021 records with 44 features. The data was collected from a large-scale, realistic testbed network deployed at UNSW Canberra, incorporating IoT, Cloud, and Edge/Fog systems, and includes both normal and attack events (e.g., DoS, DDoS, ransomware).

Following existing works [17], [18], we utilize the NetFlow version [41] of the above 4 datasets as it provides a summarized and structured representation of network traffic, which is widely used in intrusion detection, and regarded as a standard choice for evaluating AI-based security applications. By leveraging the NetFlow version, we align with existing research practices and ensure compatibility with machine learning models that require structured input data.

2) *Evaluation:* Following existing works [16], [17], we report four evaluation metrics in the experiments: macro F1 score, weighted F1 score, accuracy score, and per-class F1 score. The macro F1 score is defined as the average of the F1 scores of all classes, while the weighted F1 score is defined as the weighted average of the F1 scores of all classes. The accuracy score is defined as the ratio of the number of correctly predicted samples to the total number of samples. The per-class F1 score is defined as the F1 score of each class. Among these metrics, accuracy score and macro F1 score are selected as the main metrics to evaluate the performance of the proposed method.

3) *Dataset Details:* When handling the ToN-IoT and CSE-CIC-IDS2018 datasets specifically, we adopted a strategy of random sampling where exactly 400,000 data points were extracted from the total population. This measure was primarily aimed at maintaining data volume consistency between the

TABLE I: The performance of the compared methods on NF-CIC-IDS-2018, NF-BoT-IoT, and NF-ToN-IoT datasets in %. The **bolded** results represent the best results and underlined results represent the second-best results.

Method	NF-CIC-IDS-2018			NF-BoT-IoT			NF-ToN-IoT		
	Accuracy	Weighted F1	Macro F1	Accuracy	Weighted F1	Macro F1	Accuracy	Weighted F1	Macro F1
E-GraphSAGE [17]	43.46	54.53	41.54	81.93	80.52	41.35	59.45	62.55	31.76
M-EGraphSAGE [16]	87.57	87.60	25.90	74.91	74.90	37.30	75.17	<u>75.20</u>	33.02
Anomal-E [18]	90.05	86.79	34.31	82.68	<u>83.15</u>	56.14	65.70	61.68	26.65
UniSAGE [36]	<u>91.51</u>	<u>88.38</u>	<u>55.28</u>	82.47	82.46	55.66	<u>78.77</u>	74.25	29.25
Ours	95.01	93.56	64.74	83.44	84.59	56.45	82.31	81.55	<u>32.87</u>

TABLE II: The per-class F1 performance of the compared methods on the NF-UNSW-NB15 dataset in %. The **bolded** results represent the best results and underlined results represent the second-best results.

Method	Accuracy	Weighted F1	Macro F1	Per-class F1						
				Normal	Fuzzers	Analysis	Backdoors	DoS	Exploits	Generic
E-GraphSAGE [17]	98.44	<u>98.73</u>	<u>47.97</u>	99.70	59.29	<u>87.79</u>	<u>16.74</u>	95.64	32.59	58.54
M-EGraphSAGE [16]	98.25	98.53	35.80	99.56	63.69	68.76	3.20	93.79	0.00	29.32
Anomal-E [18]	<u>98.68</u>	98.58	44.81	99.74	64.45	81.19	6.89	96.19	<u>50.54</u>	47.18
UniSAGE [36]	98.65	98.62	41.17	99.69	<u>66.63</u>	85.19	5.93	<u>95.73</u>	0.00	<u>58.73</u>
Ours	98.76	98.86	50.75	<u>99.71</u>	69.60	88.78	18.79	96.25	60.19	62.79

datasets, thereby ensuring that the quantity of data is on a comparable scale. More importantly, this approach serves to alleviate the computational demands, granting us the ability to conduct our experiments more efficiently while limiting the necessary resources.

Finally, to maintain computational tractability and prevent excessive computational load, a maximum degree of connectivity was established for each node within the graph. We imposed an upper limit of about 5,000 on the node degrees, which is adjustable depending on the specific dataset. In instances where the degree of any given node threatened to exceed this cap, a randomized partitioning process was implemented. This process involved splitting the node in question, such that the combined total of its incoming and outgoing connections would conform to the established threshold. This threshold ensured the preservation of computational efficiency throughout the analysis, without compromising on the integrity of the network structure captured by the bipartite graph representation.

4) *Implementation Details:* Our method is implemented using Pytorch with Adam optimizer. After comparing different loss functions, we selected cross-entropy loss as our classification loss. The computational workflow commenced on a single NVIDIA RTX 3090 graphics card with 24GB memory. In the initial phase of our experiment, we structured the dataset into distinct subsets for training, validation, and testing purposes. This was accomplished by randomly partitioning the dataset at a fixed ratio of 5:2:3, respectively. Such a partitioning strategy ensures that a diverse distribution of samples is present across the different phases of model development, including training and performance evaluation.

B. Comparison with State-of-the-art Methods

To evaluate the robustness and effectiveness of the proposed E-BiGraphSAGE method, we conducted extensive experiments on four publicly available network intrusion detection datasets: NF-CIC-IDS-2018, NF-BoT-IoT, NF-ToN-IoT, and

NF-UNSW-NB15. The performance of our method was compared against several state-of-the-art approaches, including E-GraphSAGE [17], M-EGraphSAGE [16], Anomal-E [18], and UniSAGE [18]. The evaluation metrics included Accuracy, Weighted F1, Macro F1, and per-class F1 scores.

The overall results on the NF-CIC-IDS-2018, NF-BoT-IoT, and NF-ToN-IoT datasets are summarized in Table I. Our method, E-BiGraphSAGE, consistently achieved either the best or second-best performance across all three datasets, demonstrating its effectiveness in various network intrusion detection scenarios. On the NF-CIC-IDS-2018 dataset, Our E-BiGraphSAGE attained the highest Accuracy of 95.01% and a Macro F1 score of 64.74%, significantly outperforming UniSAGE, the second-best method. Similar improvement is observed on the NF-BoT-IoT dataset. On the NF-ToN-IoT dataset, our method achieved the best Accuracy and Weighted F1 scores, and almost the best Macro F1 over the second-best baselines. This performance underscores E-BiGraphSAGE's robustness in handling diverse and complex network traffic. It is worth noting that the second-best Macro F1 scores on two datasets were influenced by classes with very few samples, which can affect overall performance. Nevertheless, the experimental results confirm the outstanding performance and generalization capabilities of our algorithm across multiple datasets, reinforcing its potential as a reliable solution for real-world network intrusion detection systems.

The detailed per-class performance on the NF-UNSW-NB15 dataset is presented in Table II. Additionally, Figure 3 illustrates the confusion matrix results, offering insights into the classification performance across different classes. Our E-BiGraphSAGE model demonstrates exceptional performance on the NF-UNSW-NB15 dataset, achieving the highest scores in key metrics, including Accuracy (98.76%), Weighted F1 (98.86%), and Macro F1 (50.75%). It consistently outperforms other methods across nearly all categories, particularly in managing imbalanced classes and rare attack types, while maintaining competitive performance on normal classes. For

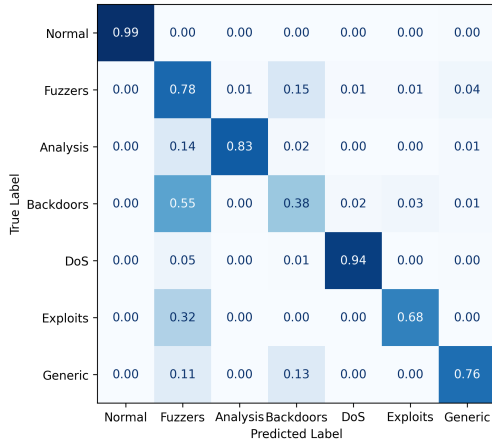


Fig. 3: Confusion Matrix on NF-UNSW-NB15

example, it achieved the best per-class F1 scores in challenging categories such as Fuzzers (69.60%), Analysis (88.78%), and Generic (62.79%), significantly surpassing methods like UniSAGE and Anomal-E. Notably, E-BiGraphSAGE also excelled in smaller attack classes, such as Backdoors, where other methods struggled. This indicates its strong capability to learn from underrepresented data, a crucial requirement for effective network intrusion detection systems. In summary, the results in Table II and the insights from the confusion matrix in Figure 3 highlight the effectiveness of E-BiGraphSAGE as a leading solution for network intrusion detection.

The above experimental results demonstrate that E-BiGraphSAGE consistently outperforms state-of-the-art methods across multiple datasets and evaluation metrics, showcasing its robustness and adaptability in diverse network intrusion detection scenarios. The significant improvements achieved by our method highlight the effectiveness of the backtracking bipartite graph learning model, which excels at capturing the directed and heterogeneous relationships inherent in network flows. This model enables a more nuanced understanding of the interactions between network entities, allowing for better differentiation between normal and malicious traffic. Additionally, the vertex splitting strategy plays a crucial role in mitigating the challenges posed by high-degree nodes, which are common in real-world network traffic. By reducing computational complexity and preventing information bottlenecks, this strategy ensures that the model remains scalable and efficient, even when applied to large-scale datasets. Together, these innovations enable E-BiGraphSAGE to achieve superior performance across both frequent and rare attack types, while maintaining high accuracy and robustness in detecting normal traffic, making it a comprehensive solution for intrusion detection.

C. Ablation Study

In Table III, we present a detailed analysis of the impact of various components of our proposed approach through ablation experiments. The baseline for our study is the basic Graph-

TABLE III: The ablation study of our proposed method on NF-BoT-IoT and NF-ToN-IoT datasets. B stands for backtracking bipartite graph learning and V denotes our vertex splitting strategy.

Method	NF-BoT-IoT		NF-ToN-IoT	
	Weighted F1	Macro F1	Weighted F1	Macro F1
Baseline	79.29	44.09	69.70	30.34
Baseline+B	84.33	54.91	74.34	30.90
Baseline+V	81.77	45.94	70.26	30.39
Ours	84.59	56.45	81.55	32.87

SAGE method applied to bipartite graphs, which serves as a foundation for evaluating the contributions of the additional components.

Firstly, we observe that when the vertex splitting strategy is applied to the baseline, there is a noticeable improvement in performance. This improvement can be attributed to the splitting of high-degree nodes, which effectively mitigates the issue of excessive smoothing of node features during the aggregation process. Secondly, the introduction of the backtracking bipartite graph learning mechanism results in a significant boost in performance compared to the baseline. This component leverages the directed relationships within the bipartite graph structure, enabling the model to better capture the flow-specific dependencies and interactions between heterogeneous vertices.

Finally, when the vertex splitting strategy is combined with the backtracking bipartite graph learning, our method achieves the best results across all metrics. The combination of these two components not only addresses the issue of excessive smoothing but also enhances the model's ability to capture the directed and heterogeneous nature of the data. This synergy between the two components demonstrates their complementary roles in improving the overall performance of the proposed method.

VI. CONCLUSION

In this paper, we propose E-BiGraphSAGE, a novel method for network intrusion detection that addresses the challenge of effective network traffic modeling. By constructing a flow-centric bipartite graph, the method effectively captures the intricate relationships within network traffic. The backtracking bipartite graph learning model enhances the representation of heterogeneous vertices, improving the detection of directed network flows. Additionally, the vertex splitting strategy reduces computational complexity, making the method more scalable and efficient. Extensive results suggest that the proposed approach holds significant potential for improving the effectiveness of network intrusion detection systems in increasingly complex network environments.

For future work, we plan to explore the integration of E-BiGraphSAGE with other advanced machine learning techniques, such as reinforcement learning and adversarial training, to further enhance the robustness, adaptability, and generalization capabilities of the proposed method.

VII. ACKNOWLEDGEMENT

This work is supported, in part, by NSFC-61872176, NSFC-62272215, Leading-edge Technology Program of Jiangsu NSF under Grant BK20202001, and National Key R&D Program of China under Grants 2020YFB1005900.

REFERENCES

- [1] K. M. Prasad, A. R. M. Reddy, and K. V. Rao, "Dos and ddos attacks: defense, detection and traceback mechanisms-a survey," *Global Journal of Computer Science and Technology*, vol. 14, no. 7-E, p. 15, 2014.
- [2] M. Uma and G. Padmavathi, "A survey on various cyber attacks and their classification," *Int. J. Netw. Secur.*, vol. 15, no. 5, pp. 390–396, 2013.
- [3] Deloitte, "2023 global future of cyber survey," 2022. [Online]. Available: https://www.deloitte.com/content/dam/assets-shared/legacy/docs/gx-deloitte_future_of_cyber_2023.pdf
- [4] A. Fleck, "Cybercrime expected to skyrocket in coming years," 2022. [Online]. Available: <https://www.statista.com/chart/28878/expected-cost-of-cybercrime-until-2027/>
- [5] F. Erlacher and F. Dressler, "Fixids: A high-speed signature-based flow intrusion detection system," in *NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2018, pp. 1–8.
- [6] A.-u.-H. Qureshi, H. Larijani, J. Ahmad, and N. Mtetwa, "A heuristic intrusion detection system for internet-of-things (iot)," in *Intelligent Computing: Proceedings of the 2019 Computing Conference, Volume 1*. Springer, 2019, pp. 86–98.
- [7] M. Masdari and H. Khezri, "A survey and taxonomy of the fuzzy signature-based intrusion detection systems," *Applied Soft Computing*, vol. 92, p. 106301, 2020.
- [8] G. Duan, H. Lv, H. Wang, and G. Feng, "Application of a dynamic line graph neural network for intrusion detection with semisupervised learning," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 699–714, 2022.
- [9] B. S. Bhati and C. S. Rai, "Analysis of support vector machine-based intrusion detection techniques," *Arabian Journal for Science and Engineering*, vol. 45, pp. 2371–2383, 2020.
- [10] X. Han, L. Xu, M. Ren, and W. Gu, "A naive bayesian network intrusion detection algorithm based on principal component analysis," in *2015 7th International Conference on Information Technology in Medicine and Education (ITME)*. IEEE, 2015, pp. 325–328.
- [11] T.-T.-H. Le, H. Kim, H. Kang, and H. Kim, "Classification and explanation for intrusion detection system based on ensemble trees and shap method," *Sensors*, vol. 22, no. 3, p. 1154, 2022.
- [12] J. Ye, Z. Feng, and X. Wang, "Flocking birds of a feather together: Dual-step gan distillation via realer-fake samples," in *2022 IEEE International Conference on Visual Communications and Image Processing (VCIP)*. IEEE, 2022, pp. 1–5.
- [13] Y. Jing, Y. Yang, X. Wang, M. Song, and D. Tao, "Turning frequency to resolution: Video super-resolution via event cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 7772–7781.
- [14] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martinez-del Rincon, and D. Siracusa, "Lucid: A practical, lightweight deep learning solution for ddos attack detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.
- [15] T.-T.-H. Le, Y. Kim, and H. Kim, "Network intrusion detection based on novel feature selection model and various recurrent neural networks," *Applied Sciences*, vol. 9, no. 7, p. 1392, 2019.
- [16] L. Chang, P. Branco, and xx, "Graph-based solutions with residuals for intrusion detection: The modified e-graphsage and e-resgat algorithms," *arXiv preprint arXiv:2111.13597*, 2021.
- [17] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-graphsage: A graph neural network based intrusion detection system for iot," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2022, pp. 1–9.
- [18] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, "Anomal-e: A self-supervised network intrusion detection system based on graph neural networks," *Knowledge-Based Systems*, vol. 258, p. 110030, 2022.
- [19] X. Jiang, R. Zhu, S. Li, and P. Ji, "Co-embedding of nodes and edges with graph neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.
- [20] L. Gong and Q. Cheng, "Exploiting edge features for graph neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 9211–9219.
- [21] S. I. Popoola, B. Adebisi, R. Ande, M. Hammoudeh, K. Anoh, and A. A. Atayero, "smote-drmn: A deep learning algorithm for botnet detection in the internet-of-things networks," *Sensors*, vol. 21, no. 9, p. 2985, 2021.
- [22] Z. Gong, J. Jiang, N. Jiang, and Y. Zhang, "An improved hybrid sampling model for network intrusion detection based on data imbalance," in *International Conference on Artificial Intelligence Security and Privacy*. Springer, 2023, pp. 164–175.
- [23] F. Xia, K. Sun, S. Yu, A. Aziz, L. Wan, S. Pan, and H. Liu, "Graph learning: A survey," *IEEE Transactions on Artificial Intelligence*, vol. 2, no. 2, pp. 109–127, 2021.
- [24] Y. Jing, Y. Liu, Y. Yang, Z. Feng, Y. Yu, D. Tao, and M. Song, "Stroke controllable fast style transfer with adaptive receptive fields," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 238–254.
- [25] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2016.
- [26] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.
- [27] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
- [28] H. Wang, M. Zhao, X. Xie, W. Li, and M. Guo, "Knowledge graph convolutional networks for recommender systems," in *The world wide web conference*, 2019, pp. 3307–3313.
- [29] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, 2020, pp. 639–648.
- [30] E. Jiawei, Y. Zhang, S. Yang, H. Wang, X. Xia, and X. Xu, "Graphsage++: Weighted multi-scale gnn for graph representation learning," *Neural Processing Letters*, vol. 56, no. 1, pp. 1–25, 2024.
- [31] Y. Jing, C. Yuan, L. Ju, Y. Yang, X. Wang, and D. Tao, "Deep graph reprogramming," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 24 345–24 354.
- [32] Z. Li, X. Shen, Y. Jiao, X. Pan, P. Zou, X. Meng, C. Yao, and J. Bu, "Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1677–1688.
- [33] Z. Wu, C. Song, Y. Chen, and L. Li, "A review of recommendation system research based on bipartite graph," in *MATEC Web of Conferences*, vol. 336. EDP Sciences, 2021, p. 05010.
- [34] J. Gao, T. Lyu, F. Xiong, J. Wang, W. Ke, and Z. Li, "Mgcn: A multimodal graph neural network for predicting the survival of cancer patients," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 1697–1700.
- [35] Z. Wang, M. Zhou, and C. Arnold, "Toward heterogeneous information fusion: bipartite graph convolutional networks for in silico drug repurposing," *Bioinformatics*, vol. 36, no. Supplement_1, pp. i525–i533, 2020.
- [36] J. Huang and J. Yang, "Unignn: a unified framework for graph and hypergraph neural networks," *arXiv preprint arXiv:2105.00956*, 2021.
- [37] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 military communications and information systems conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [38] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSP*, vol. 1, pp. 108–116, 2018.
- [39] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [40] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems," *Ieee Access*, vol. 8, pp. 165 130–165 150, 2020.
- [41] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile networks and applications*, pp. 1–14, 2022.